Factory Pattern — Class 1

Lesson 1: Why Factory Exists

Objective

By the end of this lesson, we should understand:

What problem Factory Pattern solves.

Why object creation should sometimes be separated from business logic.

How Factory connects to Strategy Pattern.

When Factory is useful in real projects.

Why It Exists

After Strategy Pattern, we already know how to remove long if-else behavior logic.

Example from Strategy:

```
NotificationStrategy strategy = strategies.get(type);
strategy.send(message);
```

Strategy answers:

Which behavior should I use?

Factory answers:

Who should create the correct object?

Factory exists because object creation can become messy when many classes are created based on conditions.

The Problem

```
public class ReportService {

    public void generate(String type) {
        Report report;

        if ("SALES".equals(type)) {
            report = new SalesReport();
        } else if ("STOCK".equals(type)) {
            report = new StockReport();
        } else if ("CUSTOMER".equals(type)) {
            report = new CustomerReport();
        } else {
            throw new IllegalArgumentException("Unsupported report type");
        }

        report.generate();
    }
}
```

What Problems Do We See?

ReportService has two responsibilities:

1. Decide which report object to create.
2. Generate the report.

This violates SRP.

It also violates OCP because every new report type requires modifying ReportService.

Lesson 2: Simple Factory

Objective

How to move object creation logic into a separate class.

Common Interface

```
public interface Report {
    void generate();
}


public class SalesReport implements Report {
    @Override
    public void generate() {
        System.out.println("Generating sales report");
    }
}


public class StockReport implements Report {
    @Override
    public void generate() {
        System.out.println("Generating stock report");
    }
}
```

Simple Factory

```
public class ReportFactory {

    public Report create(String type) {
        if ("SALES".equals(type)) {
            return new SalesReport();
        }

        if ("STOCK".equals(type)) {
            return new StockReport();
        }
```

```
        throw new IllegalArgumentException("Unsupported report type: " +
type);
    }
}
```

Service After Factory

```
public class ReportService {

    private final ReportFactory reportFactory;

    public ReportService(ReportFactory reportFactory) {
        this.reportFactory = reportFactory;
    }

    public void generate(String type) {
        Report report = reportFactory.create(type);
        report.generate();
    }
}
```

What Problem Did We Solve?

ReportService no longer knows how to create report objects.

Its responsibility becomes clearer:

```
Report report = reportFactory.create(type);
report.generate();
```

This is cleaner.

Common Mistake

Do not say Simple Factory fully solves OCP.

It improves structure, but when we add a new report type, we still modify this class:

ReportFactory

So Simple Factory is a good first step, but not the final solution.

Lesson 3: Strategy + Factory

Objective

Show us how Factory and Strategy work together.

Strategy handles behavior.

Factory selects the correct implementation.

Example

```
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}


public class AbaPaymentStrategy implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) {
        System.out.println("Pay by ABA: " + amount);
    }
}


public class AcledaPaymentStrategy implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) {
```

```
        System.out.println("Pay by ACLEDA: " + amount);
    }
}
```

Factory

```
public class PaymentStrategyFactory {

    public PaymentStrategy getStrategy(String paymentMethod) {
        if ("ABA".equals(paymentMethod)) {
            return new AbaPaymentStrategy();
        }

        if ("ACLEDA".equals(paymentMethod)) {
            return new AcledaPaymentStrategy();
        }

        throw new IllegalArgumentException("Unsupported payment method");
    }
}
```

Service

```
public class PaymentService {

    private final PaymentStrategyFactory factory;

    public PaymentService(PaymentStrategyFactory factory) {
        this.factory = factory;
    }

    public void pay(String paymentMethod, BigDecimal amount) {
        PaymentStrategy strategy = factory.getStrategy(paymentMethod);
        strategy.pay(amount);
```

```
    }
}
```

Getting Point

Strategy Pattern:

Different payment behavior.

Factory Pattern:

Create or return the correct payment strategy.

This connects naturally to our previous Strategy lesson, where we learned to remove long if-else behavior logic.

Lesson 4: Map-Based Factory

Objective

Show us how to reduce factory if-else.

Problem in Simple Factory

```
if ("SALES".equals(type)) {
    return new SalesReport();
}

if ("STOCK".equals(type)) {
    return new StockReport();
}
```

Still not beautiful.

Map Factory

```
public class ReportFactory {

    private final Map<String, Report> reports = new HashMap<>();

    public ReportFactory() {
        reports.put("SALES", new SalesReport());
        reports.put("STOCK", new StockReport());
        reports.put("CUSTOMER", new CustomerReport());
    }

    public Report getReport(String type) {
        Report report = reports.get(type);

        if (report == null) {
            throw new IllegalArgumentException("Unsupported report type: " +
type);
        }

        return report;
    }
}
```

Why This Is Better

No long `if-else`.

Cleaner lookup.

Easy to understand.

Very close to how Spring injects `Map<String, Bean>` in real projects.

Common Mistake

Do not use Map Factory when each object needs different runtime data in the constructor.

Example problem:

```
new SalesReport(startDate, endDate);
new StockReport(warehouseId);
```

In that case, we may need Supplier, Registry Factory, or Spring Bean Factory.

Lesson 5: Registry Factory

Objective

Teach us a more flexible Factory design.

Factory Using Supplier

```
public class ReportFactory {

    private final Map<String, Supplier<Report>> registry = new HashMap<>();

    public ReportFactory() {
        registry.put("SALES", SalesReport::new);
        registry.put("STOCK", StockReport::new);
        registry.put("CUSTOMER", CustomerReport::new);
    }

    public Report create(String type) {
        Supplier<Report> supplier = registry.get(type);

        if (supplier == null) {
            throw new IllegalArgumentException("Unsupported report type: " +
type);
        }

        return supplier.get();
    }
```

}

Why Supplier Is Useful

This creates a new object each time:

```
supplier.get();
```

Map Factory with object instance:

```
reports.put("SALES", new SalesReport());
```

reuses the same object.

Supplier is better when each call should create a fresh object.

Class 1 Summary

Factory Pattern is used when object creation becomes complex or duplicated.

Simple Factory moves creation logic away from business service.

Strategy + Factory is very common in enterprise projects.

Map Factory removes long if-else.

Registry Factory makes object creation more flexible.

Homework

Refactor this code:

```
public class ExportService {

    public void export(String type) {
        if ("PDF".equals(type)) {
```

```
            System.out.println("Export PDF");
        } else if ("EXCEL".equals(type)) {
            System.out.println("Export Excel");
        } else if ("CSV".equals(type)) {
            System.out.println("Export CSV");
        } else {
            throw new IllegalArgumentException("Unsupported export type");
        }
    }
}
```

Tasks:

1. Create ExportFile interface.
2. Create PdfExportFile, ExcelExportFile, CsvExportFile.
3. Create ExportFileFactory.
4. Refactor ExportService.
5. Add JsonExportFile without modifying ExportService.